Selecting Reliable LLM Reasoning with Consistency Features and Train-Fit PCA

Student ID: 3036781090

Abstract

This project studies how to select one reliable answer among four LLM candidate solutions for the same grade-school maths question. The official scorer grades only the final numeric answer, not the validity of every reasoning step. The supplied inputs are the question text, four candidate traces, and frozen whole-trace embeddings of shape [N, 4, 1024].

The final method fits PCA on training candidate embeddings (90% variance retained, k=161), concatenates the projected vectors with 17 handcrafted consistency and geometry features (339 dimensions per candidate), and trains a linear listwise softmax scorer on all training questions. At inference it always selects argmax over the four scores; no majority-vote gate is used.

The checkpoint was selected on the validation split only. Validation accuracy is 91.2281% (728/798), compared with 85.4637% for a parse-answer majority baseline. On the frozen test set the method achieves 79.4433% (371/467), compared with 69.3790% for the same baseline, a gain of about +10.1 percentage points. Uncompressed inference assets occupy approximately 0.60 MiB, and CPU evaluation with four threads finishes in well under one second on this machine. These values satisfy the limits of 16 MiB of inference assets and 30 seconds for complete CPU evaluation.

1. Task

The task is candidate selection for GSM8K-style questions under protocol `dase7506-mp2-open-hard-verifier-v5`. For each question the method returns exactly one index in {0,1,2,3}. Ranking uses selection accuracy on the complete bank; higher is better. The fixed scorer checks the selected candidate's final answer with `common.parse_answer`, including equivalent fractions and decimals. Any correct candidate earns the point.

Splits contain 4,755 / 798 / 467 train / validation / test questions and do not overlap. Every group has at least one correct final answer; test groups contain at least two different valid parsed answers. Training and method selection used the validation split; the test split was evaluated only once after freezing the checkpoint. No external datasets, additional pretrained weights, or evaluation-time LLM/API calls were used.

2. Core methods

The final student builds on a training-free majority baseline and replaces it with a train-fit linear listwise scorer over PCA-augmented features.

2.1 Majority baseline

Each candidate is parsed with `parse_answer`. If a unique modal answer exists, the first candidate with that answer is selected; otherwise the candidate closest in L2 distance to the mean of the four embeddings is chosen. This baseline needs no checkpoint and reaches 85.46% validation accuracy. Failure analysis on validation (116 errors) shows 60 unique wrong majorities and 56 tie / embedding-fallback misses. Unique

maj3 (three candidates share an answer) is about 96% reliable; maj2 is about 71%. Oracle accuracy is 100%: every question has a correct candidate, so the problem is recognition rather than generation.

2.2 Handcrafted consistency features

For each candidate the method builds a 17-dimensional vector: parseability; agreement fraction with other parsed answers; unique-mode / tied-top / singleton flags; mode strength; negative L2 distance to the mean embedding; cosine similarity to the mean; mean and max pairwise cosine to the other three candidates; text length; lexical overlap with the question; and three light interactions (mode×cos, agreement×cos, minority×cos). These features alone already beat majority voting when scored with a listwise linear model (87.09% validation).

2.3 Train-fit PCA on frozen embeddings

PCA mean and components are fit on training candidate embeddings only, with variance threshold 0.9, which yields k=161 without searching k on validation. Each candidate receives a global projection $(e - \mu)W$ and a local (within-question) projection $(e - \bar{e})W$ using the same components. Concatenating the 17 handcrafted features with both projections gives a 339-dimensional per-candidate vector. Features are standardized with training mean and standard deviation.

2.4 Linear listwise softmax scorer

A linear scorer is trained with listwise softmax cross-entropy over the four candidates of each question. When several candidates are correct, the target is uniform over the correct set. Optimization uses SGD with cosine learning-rate decay, learning rate 0.05, 300 epochs, L2 weight 1e-4, and seed 42. Training uses all training questions (not only hard non-maj3 cases). Inference always returns argmax of the four scores. A maj3 hard gate is not used in the final method: with PCA features the scorer alone outperforms gating (Section 3).

2.5 Checkpoint and interface

All inference assets live in `code/runs/model_bundle/` (`scorer.npz` and `meta.json`), about 0.60 MiB uncompressed. The public `predict(bank, arrays, checkpoint)` interface in `submission.py` loads this bundle and returns one integer index per question in bank order. Dependencies for the supplied tools and the method are limited to `numpy==2.5.3`.

3. Controlled evidence and experiments

All selection decisions used validation accuracy. Test was scored once after freezing. Ablations share the same data splits, parser, and official scorer.

3.1 Main results

Split	**Majority baseline**	**v1 (hard-only linear + maj3 gate, no PCA)**	****Final (PCA + listwise, no gate)****
Validation	85.4637% (682/798)	87.4687% (698/798)	**91.2281% (728/798)**
Test	69.3790% (324/467)	72.5910% (339/467)	**79.4433% (371/467)**

The lower absolute test accuracy is expected from the harder test construction (diverse valid answers). Relative to the majority baseline the final method gains about +5.8 pp on validation and +10.1 pp on test.

3.2 Ablations (validation)

Configuration	Val accuracy
Majority + embedding fallback	85.46%
17-D features + listwise linear (train-all, no gate)	87.09%
v1: hard-only linear + maj3 gate (no PCA)	87.47%
PCA (k=161) + listwise linear + maj3 gate	90.48%
PCA + listwise linear + no gate (final)	**91.23%**
PCA + extra “rich” text/numeric heuristics + no gate	90.98%

PCA is the main gain (about +4 pp over the no-PCA linear scorer). Removing the maj3 gate helps once the PCA scorer can correct some wrong maj3 herds without destroying easy majority-correct questions. Extra rich heuristics slightly hurt and were dropped. Across five training seeds the final configuration stays in 90.98–91.23% (seed 42 = 91.23%).

3.3 Resource measurements

Quantity	Value
Uncompressed inference assets	≈ 0.60 MiB (limit 16 MiB)
Local predict time (validation / test, 4 threads)	≈ 0.31 s / 0.21 s (limit 30 s for predict+score)
Training	CPU NumPy, minutes-scale
Seed	42

4. Rejected modifications

Each of the following was tried on validation after the final checkpoint existed. None exceeded 91.2281%, so the checkpoint was left unchanged and test was not re-scored.

Change	Val outcome
Tiny MLP (32–128 hidden) with dropout / L2 variants	Best ≈ 90.98%, below linear
Margin abstain → fall back to majority	Net harmful (breaks > fixes)
maj3 geometry flip (override when singleton is closer to mean, etc.)	≤ 91.23% or clearly worse
Hard-subset residual MLP blend	Monotone degradation with blend weight
Multi-seed score averaging; pairwise + listwise; entropy features	Flat or slightly worse

Deeper nonlinearity did not help under the asset budget: with the present features the linear listwise model is already well matched to the signal. Remaining errors look more like inseparable geometry or wrong-majority herds than underfitting.

No external data, pretrained weights, answer lookup tables, cached predictions, or evaluation-time network access were used.

5. Final result

The frozen validation score is 91.2281% (728/798). The frozen full-test score is 79.4433% (371/467). The majority baseline on the same protocol scores 85.4637% validation and 69.3790% test. Uncompressed inference assets occupy about 0.60 MiB, and complete CPU prediction with four threads finishes in well under one second on this machine. The 16 MiB asset limit and 30-second evaluation limit are met with large margins.

6. Cost, limitations, and integrity

Training uses only NumPy on CPU; a full train run is on the order of minutes on this box. Inference loads a sub-megabyte checkpoint and does not call an LLM.

Main limitations:

- The scorer grades final answers only, so the model mainly learns answer-agreement structure and embedding geometry, not step-by-step arithmetic verification. Lightweight numeric-consistency features did not lift the remaining errors enough to keep.

- Wrong maj3 herds remain hard: when three candidates share an incorrect answer, both agreement and “gold as outlier” geometry fight the correct singleton.

- Validation and test difficulty differ by construction; generalization should also be checked on any instructor audit bank using the same frozen checkpoint API.

- Capacity for larger nonlinear models is limited by the spirit of the asset budget; the MLP trials that were run did not beat the linear scorer.

7. Reproducibility and assistance

Install, train and evaluate from `code/` exactly as follows (Python 3.12+; this box used 3.13):

```
python -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
python check_data.py
python train.py --output runs/model_bundle --seed 42 --lr 0.05 --epochs 300 \
  --l2 1e-4 --pca-var 0.9 --majority-gate 0
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
python predict.py --method submission.py --checkpoint runs/model_bundle \
  --split test --output runs/test-predictions.json
python score_predictions.py --split test \
  --predictions runs/test-predictions.json --output runs/test-score.json
```

Peers may skip training and use the shipped `runs/model_bundle`. Experiment notes are in `code/runs/NOTES.md`. Repository: https://github.com/HBrEwba/DASE7506-MP2

AI assistance was used for coding support, experiment scripting, and refining the report. Method choices (majority analysis, train-fit PCA, listwise softmax, ablation decisions), measurements, and the frozen checkpoint were produced and evaluated within this project codebase. The author can explain the submitted implementation and training procedure. Course data and the fixed evaluator come from the official DASE7506-MP2 package (GSM8K / Qwen3-0.6B candidates and embeddings; see `code/data/LICENSE.txt` and `NOTICE.txt`).

References

[1] Karl Cobbe et al. “Training Verifiers to Solve Math Word Problems.” arXiv:2110.14168, 2021. (GSM8K)

[2] Qwen Team. “Qwen3 Technical Report.” arXiv:2505.09388, 2025. (candidate traces prepared with Qwen3-0.6B; revision recorded in the data manifest)

[3] Harold Hotelling. “Analysis of a Complex of Statistical Variables into Principal Components.” Journal of Educational Psychology, 24(6):417–441, 1933.

[4] Xudong Wu. DASE7506 Mini-Project 2: Selecting Reliable LLM Reasoning. Course materials, Fall 2026. https://github.com/xudongwu-0/DASE7506-MP2